FREIE UNIVERSITÄT BERLIN

Programmierung in C
Einführung

Dr. Larissa Groth, Jonathan Biertimpel
Computer Systems & Telematics (CST)

Vorstellungsrunde

- Name
- Studiengang
- Schon mal programmiert?
- Schon mal C programmiert?
- Was erwarten Sie vom C-Kurs?



Rückblick: erstes C-Programm

hello.c — Code bearbeiten

```c
#include <stdio.h>

int main(void) {
    printf("Hllo Wrld\n");
    return 0;
}
```

Ausgabe ansehen | Gerüst zurücksetzen

DEINE AUSGABE · SIMULATION

```
Hllo Wrld
```

Die Vorschau zeigt deinen eingegebenen Text. Vergleiche ihn mit Hello World und lass den Zeilenumbruch \n stehen.

rechnen.c — Code bearbeiten

```c
#include <stdio.h>

int main(void) {
    int a = 10;
    int b = 3;

    printf("Summe: %d\n", a+b);
    printf("Differenz: %d\n", a-b);
    printf("Produkt: %d\n", a*b);
    printf("Quotient: %d\n", a%b);
    return 0;
}
```

Ausgabe ansehen | Gerüst zurücksetzen

DEINE AUSGABE · SIMULATION

```
Summe: 13
Differenz: 7
Produkt: 30
Quotient: 1
```

Vergleiche die vier Ergebnisse mit der Soll-Ausgabe für a = 10 und b = 3. Prüfe Werte, Operatoren und Zeilenumbrüche.

Teilen ganzer Zahlen und Modulo

a = 10;

b = 3;

- Mathematisch: 3,33
- Ganzzahldivision: a/b = 3
- Modulo: a%b = 1

Organisation

- „Selber denken macht schlau“
- Materialien und Aufgaben auf der Kurswebsite
  - Bearbeitung im eigenen Tempo
  - Gemeinsame Zwischenziele als Orientierung
- Zeit zur Bearbeitung der Aufgaben
- Individuelle Unterstützung bei Fragen und Problemen

Dr. Larissa Groth (she/her)
larissa.groth@fu-berlin.de
Office: Takustr. 9, 157

Jonathan Biertimpel (he/him)
Jonathan.biertimpel@fu-berlin.de
Office: Takustr. 9, 164

Office hours by appointment

Die Aufgaben

Datentypen

- Lernziele
- Theorie und Hintergrund
- Materialien
- Aufgaben
- Hinweise
- Selbstkontrolle

Lernstandschecks und Abschluss

- Lernstandschecks I und II zur Überprüfung der eigenen Kenntnisse
- Jeweils 90 Minuten zur selbstständigen Bearbeitung
- Gezielte Wiederholung bei Lücken

- Projekt in C: Entwurf, Implementierung und Präsentation
- Nachvollziehbare eigene Beiträge und Verständnis des Codes

Lernziele

Sie erläutern:

- die Begriffe Programm, Programmiersprache und Algorithmus
- Unterschiede zwischen Python, Java, C und Assembler an einem Beispiel

Sie beschreiben:

- die zusätzliche Kontrolle und Verantwortung beim Programmieren in C
- den Weg vom C-Quelltext zum ausführbaren Programm

Sie verwenden:

- ein vorgegebenes Makefile zum Übersetzen eines C-Programms

Definitionen

Programm und Programmiersprache

- Programm:
  - Anweisungen zur Verarbeitung von Daten
  - Beschreibung der Operationen, Operanten und ihres Ablaufs

- Programmiersprache
  - Formale Sprache zur Beschreibung von Programmen
  - Syntax: Regeln für die zulässige Schreibweise
  - Semantik: Bedeutung der Anweisungen

Algorithmus

- Endliche Beschreibung einer Vorgehensweise zur Lösung einer Aufgabe
  - Ausführbare Einzelschritte
  - Eindeutig festgelegter Ablauf
  - Deterministischer Algorithmus: gleicher Input, gleicher Output

- Denken Sie an ein Kochrezept:
  - Zutaten und Mengen als Eingaben
  - Zubereitungsschritte als Verarbeitung
  - Fertiges Gericht als Ergebnis
  - Warum wird das Essen nicht immer gleich?
    - Unterschiedliche Bedingungen

Diskussion: Präzision von Anweisungen

Anweisung im Rezept:

„Backen Sie den Kuchen, bis er fertig ist.“

Welche Angaben fehlen?

- Welche Voraussetzungen setzt die Anweisung voraus?
- Wie lässt sich ein überprüfbares Ende formulieren?

Kurzer Austausch zu zweit, anschließend gemeinsame Sammlung

Beispiel: Addition zweier Zahlen

Problem

- Summe zweier ganzer Zahlen berechnen

Algorithmus

Beispiel

Eingaben: 7 und 5.

Ausgabe: 12.

Beispiel: Addition zweier Zahlen

Problem

- Summe zweier ganzer Zahlen berechnen

Algorithmus

1. Erste Zahl einlesen
2. Zweite Zahl einlesen
3. Summe berechnen
4. Summe ausgeben

Beispiel

Eingaben: 7 und 5.

Ausgabe: 12.

Vergleich (mehr oder weniger) beliebter Programmiersprachen

Sprachvergleich: dieselbe Berechnung

Python

```
a = 5
b = 7
print(a + b)
```

Java

```
int a = 5;
int b = 7;
System.out.println(a+b);
```

C

```
int a = 5;
int b = 7;
printf("%d\n", a+b);
```

Assembler

```
mov eax, 5
mov ebx, 7
add eax, ebx

# Ergebnis in Text umwandeln (hier ausgelassen)

mov rax, 1                # Systemaufruf write
# weitere Parameter setzen und Adresse des Textes angeben

syscall
```

Assembler

Beispielausschnitt für x86 in Intel-Schreibweise

```
mov eax, 7
add eax, 5
```

Bedeutung

- Register eax: kleiner Speicherplatz im Prozessor
- mov: Wert 7 im Register ablegen
- add: Wert 5 addieren. Ergebnis im Register: 12

Ein Assembler übersetzt die Befehle in Maschinencode.

Abstraktion und Kontrolle

Sprachen übernehmen unterschiedliche Prüfungen und Verwaltungsaufgaben.

Sprache	**Typisierung**	**Dynamischer Speicher**
Python	dynamisch	automatische Verwaltung
Java	statisch	automatische Verwaltung
C	statisch	explizit reservieren und freigeben

Statisch: Typverträglichkeit vor allem beim Übersetzen prüfen

Dynamisch: Typverträglichkeit vor allem zur Laufzeit prüfen

Motivation zur Programmierung in C

Motivation für C

- Daten und Speicher
  - Datentypen, Datenrepräsentation und Adressen
  - Bewusster Umgang mit dynamisch reserviertem Speicher
- Programmstruktur
  - Zerlegung in Funktionen und Quelldateien
  - Schnittstellen für eigene und fremde Komponenten
- Anwendungsbereiche
  - Systemsoftware, eingebettete Systeme und native Bibliotheken

Bibliotheken und nativer Code

Bibliothek: Sammlung nutzbarer Funktionen

Beispiel: math.sqrt in CPython

```
import math
print(math.sqrt(9))
```

Python-Schnittstelle

- Aufruf math.sqrt(9), Ergebnis: 3.0

Native Implementierung

- Das Modul math nutzt überwiegend die C-Mathematikbibliothek der Plattform.

Aufbau eines C-Programms

```
#include <stdio.h>

int main(void) {
    printf("%d\n", 7 + 5);
    return 0;
}
```

Ausgabe: 12

stdio.h

- Deklaration von printf

main

- Programmeinstieg

printf

- Ausgabe der Summe

return 0

- Erfolgreicher Abschluss

Übersetzen und Ausführen

Dateien und Werkzeuge im einfachen C-Projekt

Schritt	**Werkzeug oder Aufruf**	**Ergebnis**
Bearbeiten	Editor	Quelltextdatei rechner.c
Übersetzen	Compiler und Linker	ausführbare Datei rechner
Ausführen	./rechner	Ausgabe: 12

Änderungen am Quelltext erfordern eine erneute Übersetzung.

Makefiles

Vorgegebenes Makefile

- Beschreibung der Befehle und Dateiabhängigkeiten für die Übersetzung
- Zunächst Nutzung bereitgestellter Makefiles

Übersetzen im Projektverzeichnis

make

Programm unter Linux/Unix starten

./rechner

Eigene Makefiles folgen im späteren Kapitel.

Aufgabe

1. Starten Sie ein Terminal im Projektordner

2. Führen Sie folgenden Befehl aus:
   make

3. Starten Sie das Programm:
   ./rechner

4. Starten Sie einen Editor mit beispielsweise:
   nano rechner.c

5. Ändern Sie die Rechnung und speichern Sie.

6. Führen Sie erneut das Programm aus (./rechner)
   → Was beobachten Sie?

Diskussion: Änderung des Quelltexts

Was haben Sie beobachtet?

Welche Themen werden wir behandeln?

Block I:

1. Einführung
2. Variablen, Datentypen und Berechnung
3. Bedingungen und Schleifen
4. Programmzerlegung, Funktionen und Debugging

Block II:

5. Manpages
5. Arrays, Zeichenketten und Listen
5. Pointer und Datenaustausch
5. Dynamischer Speicher und Strukturen
5. Makefiles
5. Header, Quelldateien und Schnittstellen

Block III:

11. Projekt

Welche Themen werden wir behandeln?

Block I:

1. Einführung
2. Variablen, D
3. Bedingunge
4. Programmze

Block II:

5. Manpages
5. Arrays, Zeic
5. Pointer und
5. Dynamische
5. Makefiles
5. Header, Que

Block III:

11. Projekt



Dieses Thema wird gerade vorbereitet. Bitte arbeiten Sie zunächst an den bereits veröffentlichten Aufgaben weiter.

Zur Kursübersicht

Fragen?